oneDNN Updates

Dmitry Zarukin, Intel
2026/2/25

UXL
Unified Acceleration Foundation

Agenda

- RISC-V
- Weights Only Quantization
- MXFP8 and NVFP4 Quantization
- Asynchronous Threadpool Runtime
- Mixture of Experts (MoE)
- Host-side scalars

RISC-V Contributions

RISC-V is an open-standard Instruction Set Architecture (ISA) based on Reduced Instruction Set Computer (RISC) principles.

Since March 2025 oneDNN got traction on RISC-V front.
New maintainers:
Fei Zhang (@zhangfeiv0) from ISCAS
Jian Zhang (@zhangjian29) from ZTE Corporation

Available support (f32/bf16/f16):

- Convolution
- Matmul / Inner Product
- Binary / Eltwise
- Normalizations
- Softmax
- Pooling

Weights Only Quantization (WOQ)

Weights only quantization is a memory saving technique for GeMM operation when weights tensor (or matrix B) is passed as int8 or int4 memory buffer (opposed to f16/bf16/f32) with supportive scales and zero-points quantization buffers.

- oneDNN introduced new data types (s4, u4), extended fpmath-mode attribute - to allow floating-point tensor matrix multiplied by integer - and extended quantization API with:
  - Data types for scales and zero-points;
  - Groups which represent the number of elements sharing a single scale/zero-point value.
- The technique is very popular in LLM models across various frameworks to save on huge weights tensors and improve operation bandwidth.
- Introduced in oneDNN v3.8 and got extensions in following versions.
- Performance benefit over floating-point weights case highly depends on problem sizes, hardware and quantization parameters.
- Example[1]

[1]:https://uxlfoundation.github.io/oneDNN/page_matmul_with_weight_only_quantization_cpp.html#doxid-matmul-with-weight-only-quantization-cpp

UXL
Unified Acceleration Foundation

MXFP8/4 and NVFP4 Quantization

MXFP8 and MXFP4 describe Micro-scaling techniques over FP8 and FP4 data types.

NVFP4 describes a new data type additionally to scaling techniques associated with it.

- oneDNN introduced new data types (e5m2, e4m3, e8m0, e2m1) and scaling API (quantization_mode::dynamic_mx and dynamic_fp) to support the flows.
- Dynamic kinds of scaling API allows to mark destination scales as “dynamically” computed by oneDNN and stored in the user’s output for future use.
- Introduced in oneDNN v3.10 and v3.11.
- Performance benefit over higher floating-point precision case highly depends on problem sizes and hardware.
- Example[1]
- Documentation with formulae[2]

[1]:https://uxlfoundation.github.io/oneDNN/page_mxfp_matmul_cpp.html#doxid-mxfp-matmul-cpp
[2]:https://uxlfoundation.github.io/oneDNN/dev_guide_attributes_quantization.html#dynamic-quantization

Asynchronous Threadpool Runtime

oneDNN's Threadpool interface provides flags to mark threadpool implementation to be synchronous or asynchronous.

Asynchronous implementation means parallel tasks execution is untied to parallel task submission which makes it like other public GPU runtimes available on the market but without additional software providing scheduling and device interoperability.

- The request came from Google's XLA runtime of openXLA Project.
- The motivation for asynchronous runtime implementation has better resource utilization.
- It puts strict requirements on internal objects lifetime management inside the implementation.
- oneDNN introduced a new "wait()" call in the threadpool interface to synchronize on a threadpool before proceeding further.
- Complete support for x86 introduced in oneDNN v3.11. Other backends require engineering investment for being supported.
- On its way to XLA runtime upstream.
- Performance benefit over synchronous case is around 10%. Model-wise the benefit from new runtime might reach up to 40-60%.

Mixture of Experts

Mixture of Experts (MoE) is an optimization technique that boosts efficiency by using a gating network to route inputs to specialized subnetworks ("experts") rather than activating a full model.

- oneDNN introduced a new memory descriptor, which expresses a complex activation input/output, to invoke a part of MoE flow known as grouped_mm[1].
- Processes device buffers of concatenated tokens post routing and offsets to each expert's data.
- Supports classic floating-point mode and Weights Only Quantization (WOQ).
- Targets oneDNN v3.12.

[1]: https://docs.pytorch.org/docs/stable/generated/torch.nn.functional.grouped_mm.html

Host-side scalars

They are single numerical values that reside and are managed in the CPU (or host) memory rather than the GPU (or device) memory.

- oneDNN introduced a new memory descriptor and a new memory constructor to use host-side scalars mainly targeting quantization parameters (per tensor scale and zero-point).
- Allow users to save on device buffer allocation and data copy to those buffers.
- Introduced in oneDNN v3.10.

Thank you!

UXL
Unified Acceleration Foundation

Backup: Weights Only Quantization

A	K							
M	0	1	2	3	4	5	6	7
	8	9	10	11	12	13	14	15
	16	17	18	19	20	21	22	23
	24	25	26	27	28	29	30	31
	32	33	34	35	36	37	38	39
	40	41	42	43	44	45	46	47

B	N															
K	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47
	48	49	50	51	52	53	54	55	56	57	58	59	60	61	62	63
	64	65	66	67	68	69	70	71	72	73	74	75	76	77	78	79
	80	81	82	83	84	85	86	87	88	89	90	91	92	93	94	95
	96	97	98	99	100	101	102	103	104	105	106	107	108	109	110	111
	112	113	114	115	116	117	118	119	120	121	122	123	124	125	126	127

$$C_{mn} = \sum_{j=0}^{N} \sum_{i=0}^{G} a_{mi} * (scale_{g_j} * (b_{in} - zp_{g_j}))_{f16}, where$$

N – *is the number of groups,*

G – *is the size of the group.*

scales	N															
gK	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31

UXL
Unified Acceleration Foundation

Backup: MXFP8/4 and NVFP4 Quantization

Dynamic Quantization

oneDNN supports two dynamic quantization modes, for scales only (no zero-point) following the formula:

$$x_{f32}[:] = scale_x \cdot x_{quant}[:]$$

where x_{f32} and x_{quant} are the non-quantized and quantized representation of x respectively, and $scale_x$ is a scaling factor.

When using `quantization_mode::dynamic_mx`, $scale_x$ is computed following the OCP MX Formats Specification, namely $scale_x$:

- has `e8m0` datatype,
- is computed for each group of size `32`,
- is computed as the largest power-of-two less than or equal to the maximum absolute value of the group divided by the largest power-of-two representable in the x_{quant} datatype, e.g., $E8M0(amax(x_{quant}[:]))/E8M0(MAX_QUANT_DT)$.

When using `quantization_mode::dynamic_fp`, $scale_x$ is computed in `f32` first and then converted to a scale datatype, namely $scale_x$:

- has `f8_e4m3` datatype,
- is computed for each group of size `16`,
- is computed as $SCALE_DT(amax(x_{quant}[:])/MAX_QUANT_DT)$.

UXL
Unified Acceleration Foundation

Backup: Asynchronous Threadpool

- Backend implementation efforts:
  - Update backend parallel calls lambda functions to capture-by-copy, otherwise, internal objects would expire before execution task finished and associated changes related to that (code order, copy of uncopied objects, etc.)
  - Update usage of complex stack objects, e.g., brg_matmul_exec_ctx_t – either wrap into std::shared_ptr, which prolongs life of managed object given lambda capture gets updated, or replace them with something else.
  - Any memory buffer accesses must be moved inside parallel section, e.g., DEFINE_ARG_SCALES_BUFFER macro, or precompute_scales(…) function, or serialized reduction after a parallel call.